QWEN

Helix Constitution — Universal QWEN.md

Field	Value
Revision	8
Created	2026-05-20
Last modified	2026-05-29T00:00:00Z
Status	active
Status summary	Mirrored Constitution.md §11.4.102 (Mandatory systematic-debugging activation + always-loaded skill-discovery + plugin-dependency availability, User mandate 2026-05-29) into this QWEN.md. §11.4.78–§11.4.101 mirrors continue from earlier Revisions.
Issues	none
Issues summary	—
Fixed	§11.4.102 mirror
Fixed summary	§11.4.102 lands in lockstep with the Constitution.md §11.4.102 addition.
Continuation	—

Table of contents

- How inheritance works
- Identity & posture
- Top-level invariants for every agent

- Critical base rules restated (for agents that don’t honour @imports)
  - §11.4 — Anti-bluff covenant — END-USER QUALITY GUARANTEE
  - §1.1 — Mutation-paired gates
  - §11.4.10 — Credentials-handling mandate
  - §11.4.17 — Universal-vs-project classification
  - §11.4.20 — Subagent-driven-by-default
  - §11.4.73 — Main-specification document versioning + revision discipline
  - §11.4.74 — Submodule-catalogue-first discovery + extend-don’t-reimplement
  - §11.4.76 — Containers-submodule mandate
- Companion documents

How inheritance works

This `QWEN.md` is for the **Qwen Code CLI agent** (`qwen-code`). It documents the universal Helix Constitution from the Qwen agent’s perspective — the same content seen by Claude Code via `CLAUDE.md` and by OpenCode / Cursor / generic AI tooling via `AGENTS.md`.

The **authoritative source** is `Constitution.md` in this repository. When this file diverges from `Constitution.md`, the latter wins. This file exists because:

1. Qwen Code does not always honor `@import` directives across files; restating the critical rules inline ensures the agent reads them on every session.
2. Cross-agent parity: Claude Code (CLAUDE.md), OpenCode/Cursor (AGENTS.md), Qwen Code (QWEN.md) all start from the same constitutional baseline.

Discover this file via the canonical parent-walk pattern documented in every consuming project’s `CLAUDE.md` / `AGENTS.md`:

```
find_constitution_root() {
    local cur="${1:-$PWD}"
    while [[ "${cur}" != "/" ]]; do
        if [[ -f "${cur}/constitution/Constitution.md" ]]; then
            echo "${cur}/constitution"; return 0
        fi
        cur="$(dirname "${cur}")"
    done
    return 1
}
```

Identity & posture

When the Qwen Code agent loads this file as part of session bootstrap, it operates under the Helix Constitution with the same identity, anti-bluff posture, and end-user quality covenant as any other CLI agent in the catalogue. There is no “Qwen-lite” interpretation — Qwen Code is held to the full §11.4 covenant.

Top-level invariants for every agent

1. **Read order on a cold start.** `CLAUDE.md` / `AGENTS.md` / `QWEN.md` (this file) for the AI-agent posture; `Constitution.md` for the canonical universal rules; then the consuming project’s `CLAUDE.md` for project-specific extensions.
2. **Multi-mirror push.** Every commit on this submodule MUST land on all four canonical mirrors (GitHub + GitLab + GitFlic + GitVerse) per §2.1.
3. **Anti-bluff is non-negotiable.** Passing tests MUST imply working features. Bluffing PASSes (and FAILs that hide root cause) is a release blocker (§11.4 + §11.4.1).
4. **Submodule-catalogue-first.** Before scaffolding anything, survey the `vasic-digital` + `HelixDevelopment` orgs for an existing Submodule (§11.4.74). Reuse → extend → no-match in that order.
5. **Containers-submodule for ALL container workloads.** Use `vasic-digital/containers` (§11.4.76) — never reinvent compose/ boot orchestration in-project.

Critical base rules restated (for agents that don’t honour @imports)

§11.4 — Anti-bluff covenant — END-USER QUALITY GUARANTEE

Forensic anchor — verbatim user mandate (2026-04-28, reasserted 2026-05-21, 2026-05-29):

> “all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

Operative rule. Captured tests MUST exercise the behavior they claim to verify. PASS-without-execution paths, mock-only tests that don't round-trip, skip-by-default integration tests that mask real failures, metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-without-runtime PASS are all critical defects regardless of how green the summary line looks. Each test failure in CI MUST imply a real broken feature; each PASS MUST imply the feature actually works for the end user. Tests and HelixQA Challenges are bound EQUALLY.

The verbatim covenant MUST be present in every consumer governance file (project Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md) and in every owned submodule's equivalent. Tools that don't expand `@imports` still read the literal text.

§1.1 — Mutation-paired gates

Every captured-evidence gate MUST ship with a paired mutation test that mutates the gate's anchor + runs the gate + asserts the gate now FAILs. Absence of the paired mutation is itself a bluff.

§11.4.10 — Credentials-handling mandate

Credentials are NEVER committed to git, NEVER logged, NEVER printed to stdout/stderr. `.env` files are `.gitignored`; committed siblings are `.env.example` placeholders only. Pre-store leak audit (§11.4.10.A) scans every PR for credential patterns before merge.

§11.4.17 — Universal-vs-project classification

Every new rule MUST declare itself **universal** (applies to every project consuming this Constitution) or **project** (applies only to one specific project). Misclassification (universal rule landing in a project's local CLAUDE.md instead of the constitution submodule) is a release blocker.

§11.4.20 — Subagent-driven-by-default

When a CLI agent (Qwen Code included) has a subagent / Task tool available, the default mode of operation is to dispatch subagents for discrete units of work rather than perform them inline. This protects context, enables parallel work, and matches the §11.4.27 no-fakes-beyond-unit-tests posture (each subagent runs real tools).

§11.4.73 — Main-specification document versioning + revision discipline

Projects with a main specification (`docs/specs/.../specification.V<N>.md`-shaped) maintain TWO version axes:

- **Primary** (V1 / V2 / V3 …) bumps for major rewrites only.
- **Secondary** (`Revision N` in the metadata table) bumps for additive changes within a primary version.

Spec edits trigger mandatory comprehensive planning and implementation in the consuming project — they are not isolated doc tweaks (the “spec ripple” rule).

§11.4.74 — Submodule-catalogue-first discovery + extend-don’t-reimplement

Direct user mandate (verbatim, 2026-05-20): “We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in `vasic-digital` and `HelixDevelopment` organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times. For any missing features we MUST IMPLEMENT them properly and extend those Submodules further!”

Before scaffolding any new module / package / helper / utility, the Qwen Code agent MUST: (1) survey `vasic-digital` + `HelixDevelopment` on GitHub + GitLab — the canonical inventory is `submodules-catalogue.md` (142 repos categorised); (2) reuse an existing Submodule when it covers ≥ 80%; (3) extend in-place via upstream PR when 80%+ matches; (4) document the survey result via `Catalogue-Check: reuse|extend|no-match <org/repo>@<sha>` in the tracker row.

Every Submodule in the catalogue is subject to the same development-cycle rules (§11.4 anti-bluff, §1.1 paired mutations, §11.4.10 credentials, §11.4.61 metadata + ToC, §11.4.65 universal export, §11.4.73 spec versioning, §2.1 multi-mirror push, §3 propagation order).

§11.4.76 — Containers-submodule mandate

Direct user mandate (verbatim, 2026-05-20): “For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemy / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: `https://github.com/vasic-digital/containers` (`git@github.com:vasic-digital/containers.git`). Containers Submodule contains all means for us to Containerize our code and services! If any feature or

Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!”

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), the Qwen Code agent MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule when scaffolding the consuming project; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via the Submodule’s `pkg/boot` + `pkg/compose` + `pkg/health` APIs so the operator is never required to start `podman machine` / `docker compose up` manually — the boot is part of the test entry point (**on-demand-infra invariant**); (4) extend the Submodule via upstream PR when a runtime / lifecycle primitive is missing — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule. Short-circuit fakes that bypass boot are a §11.4 violation. A passing test MUST imply the infra was up.

Tracker rows touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha>` (or `reuse`); `no-match` requires demonstrating the Submodule cannot model the workload.

Planned anti-bluff gate `CM-CONTAINERS-USED` scans container-touching PRs for `digital.vasic.containers/...` imports. Paired mutation strips the import + asserts FAIL.

Canonical authority: `Constitution.md` §11.4.76.

Non-compliance: reinventing compose orchestration in-project is a release blocker.

§11.4.77 — Regeneration-mechanism-required mandate

Direct user mandate (verbatim, 2026-05-20): “We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content! Add this mandatory safety rule / constraint into ours root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md or any other required document / file from the constitution Submodule!”

Forensic anchor. 2026-05-20T15:00Z: ATMOSphere parent’s audio Tier 1 `commit_all.sh` stalled 4 h on `git add -A` scanning 274 GiB `.git-backup-*` + 159 GiB `RKTools/linux/` + 167 GiB `qa-results/`

— all untracked but un-gitignored. Bare `.gitignore` fix would orphan every fresh clone (missing RKTools, missing test infra, missing build outputs).

Every `.gitignore` entry the Qwen Code agent considers adding that excludes (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either **re-obtain** (vendor tarball, SDK installer, package registry, git submodule, object store) OR **re-generate** (build pipeline, code-gen, asset render, captured-evidence replay, container build).

Required artefacts per qualifying `.gitignore` entry: (1) `.gitignore-meta/<entry-slug>.yaml` declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) post-clone bootstrap entry (`scripts/setup.sh` or canonical equivalent) running the mechanism non-interactively; (3) pre-build gate verifying regenerated content present OR stamp `.gitignore-meta/.regenerated/<slug>.ok` recent; (4) README + `docs/guides/*.md` describing the mechanism + manual fallback + time/disk budget + per-§11.4.10 credentials.

No escape hatch: bare `.gitignore` additions without the mechanism are §11.4 PASS-bluff variants — codebase appears complete but fresh clone cannot build / run. No `--skip-regen-mechanism`, `--gitignore-is-enough`, `--operator-already-has-content` flag.

Planned anti-bluff gate `CM-GITIGNORE-REGEN-MECHANISM` scans every `.gitignore` addition for matching `.gitignore-meta/` sibling. Paired §1.1 mutation strips a required YAML key → gate FAILs.

Composes with §11.4.6, §11.4.65, §11.4.66, §11.4.71, §11.4.74, §11.4.75, §11.4.76, §9 / §9.2, §3.

Canonical authority: `Constitution.md` §11.4.77.

Non-compliance is a release blocker regardless of context.

§11.4.78 — CodeGraph code-intelligence mandate

Direct user mandate (verbatim, 2026-05-20): “Make codegraph MANDATORY CHOICE for this purpose for all of our project … All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!”

Every consuming project worked on by AI coding agents — Qwen Code included — MUST install, initialize, and use **CodeGraph** (`https://github.com/colbymchenry/codegraph`, npm `@colbymchenry/codegraph`): a local SQLite semantic code-knowledge-graph exposed to agents over MCP, 100% local. Install globally via npm

(no `sudo`). `codegraph init` + `codegraph index`: `.codegraph/config.json` is tracked, `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the `exclude` list MUST exclude other-owned submodules and every §11.4.10 credential/secret path. Wire the `codegraph serve --mcp` server into every CLI agent the developers use — for Qwen Code via `.qwen/settings.json` (`qwen mcp add codegraph codegraph serve --mcp --scope project --transport stdio`). Cover the integration with an anti-bluff verification suite using an unforgeable per-agent challenge (a fact obtainable only by calling a CodeGraph MCP tool); un-runnable agents are documented SKIP gaps, never faked PASSes. Document in `docs/CODEGRAPH.md`. CodeGraph is consumed as the npm package (§11.4.74), not a git submodule.

Composes with §11.4.3, §11.4.10, §11.4.12, §11.4.65, §11.4.30, §11.4.74, §11.4.77, §11.4, §1.1.

Canonical authority: `Constitution.md` §11.4.78.

Non-compliance is a process violation; a project worked on by AI agents without CodeGraph installed, wired, and anti-bluff-verified is in breach.

§11.4.79 — Own-org submodules MUST be included in the CodeGraph index

Direct user mandate (verbatim, 2026-05-21): “All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!”

Refines §11.4.78 step 2 with a per-submodule-ownership split. **Own-org submodules** — full write access via the project’s CLIs (canonical orgs: `vasic-digital` + `HelixDevelopment`) — MUST be INCLUDED in the index so Qwen Code (and every other AI agent) sees a unified call-graph across the project’s domain + own-org infra code. **Third-party submodules** (the §11.4.74 `no-match → vendor` path) MUST be EXCLUDED. Operational steps: (1) `git submodule update --remote --merge` to pull latest before re-indexing — respect load-bearing third-party pins. (2) Adjust `.codegraph/config.json` exclude list. (3) Re-index via `scripts/codegraph_setup.sh`. (4) Validate via `scripts/codegraph_validate.sh` probe resolving a symbol that lives ONLY inside an own-org submodule. (5) Paired §1.1 mutation: temporarily add own-org path to exclude → validate FAILs → restore.

Composes with §11.4.74, §11.4.78, §11.4.10, §1.1, §107.

Canonical authority: `Constitution.md` §11.4.79.

Non-compliance is a process violation; severe cases (own-org submodules silently excluded without an audit trail in `.codegraph/config.json`) are release blockers.

§11.4.80 — CodeGraph regular-update + sync automation mandate

Direct user mandate (verbatim, 2026-05-21): “We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed […] Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule […] Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents […]”

Three deliverables in this constitution submodule: (1) `scripts/codegraph_update.sh` — npm-installs latest; §107 anti-bluff verifies `codegraph --version` reflects the new version. (2) `scripts/codegraph_sync.sh` — runs `status` → `sync` → `status` → project’s validate; appends to both project’s + constitution’s `docs/codegraph/Status.md`. (3) `docs/codegraph/Status.md` + `Status_Summary.md` ledgers, exported per §11.4.65 to `.html` + `.pdf`.

Cadence: weekly floor (§11.4.45). Scripts inherited by reference (§3) — Qwen Code consuming projects invoke them at `${CONST_DIR}/scripts/codegraph_*.sh`, never copy.

Composes with §11.4.78, §11.4.79, §11.4.10, §11.4.45, §11.4.65, §107, §1.1.

Canonical authority: `Constitution.md` §11.4.80.

Non-compliance is a process violation; severe cases (>2 weeks since last update + open AI-agent work) are release blockers.

§11.4.81 — Cross-platform-parity mandate (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): “Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!”

Every consuming project whose supported-platforms manifest lists more than one OS MUST ship per-OS-equivalent implementations + tests for every feature/gate/challenge/mutation that depends on platform-specific primitives. Runtime dispatch via `uname -s` (or equivalent platform detection). Three sub-mandates: **(A)** Per-OS implementation REQUIRED — cgroup/systemd//`proc` primitives

have documented equivalents (POSIX `setrlimit`/`ulimit`, launchd, `rctl`, Job Object). **(B)** Per-OS tests REQUIRED — every gate test has `case "$(uname -s)" in` branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch; SKIP-with-reason only when the platform genuinely cannot enforce. **(C)** Honest kernel-gap citation + adjacent equivalent test REQUIRED where no equivalent exists (canonical: XNU does NOT enforce `RLIMIT_AS` for unprivileged processes → SKIP with exact reproducer + adjacent test of what IS enforced, e.g. `RLIMIT_CPU+SIGXCPU`). The adjacent test is itself anti-bluff per §11.4 with a paired §1.1 mutation.

Per-OS equivalence catalogue (canonical): `systemd-run --user --scope` ↔ POSIX `ulimit -t -u` / launchd; cgroup `MemoryMax` ↔ XNU gap (use `RLIMIT_CPU` adjacent) / `rctl` / Job Object; cgroup `TasksMax` ↔ `RLIMIT_NPROC`; `/proc/<pid>/oom_score_adj` ↔ no Darwin/BSD equivalent.

Composes with §11.4.1 / §11.4.2 / §11.4.3 (strictened — SKIP only when kernel cannot) / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.27 / §11.4.69 / §11.4.70 / §107. Pre-build gate `CM-CROSS-PLATFORM-PARITY` + paired §1.1 mutation. No escape hatch.

Canonical authority: `Constitution.md` §11.4.81.

Non-compliance is a release blocker on multi-platform projects.

§11.4.82 — Iteration-speedup discipline mandate (User mandate, 2026-05-22)

Direct user mandate (verbatim, 2026-05-22): “How can we speed-up this whole development and fixing process? … all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md!”

Iteration cycle time bounds the project’s defect-discovery rate. Slow cycles ship fewer validated features per unit of operator time. The 2026-05-22 forensic session witnessed ~3 hours of operator wait on a job whose intrinsic compute was ~90 min — every wasted minute came from a missing speedup discipline.

Every consuming project’s build / test / commit / debug pipeline MUST adopt the following 9 speedup disciplines AS MANDATORY:

- **(A) Phase 1 forensic before any speculative source patch.** Speculative patches without root-cause evidence are §11.4.6 + §11.4.82 violations.
- **(B) Live-ADB-First (or live-equivalent) before any rebuild.** §11.4.51 strengthened to release-blocker. Skipping live-probe for LIVE_ADB_TESTABLE changes = §11.4.82 violation.

- **(C) Pre-flight before rebuild orchestrators.** 30-sec readiness check verifies device + sink reachability, memory budget, lock files, orphan processes.
- **(D) Persistent build caches outside containers.** `ccache` + Soong / sccache / Gradle daemon state bind-mounted to host. Subsequent rebuilds drop from ~40 min to 5-15 min.
- **(E) Module-only rebuild for `CONFIG_*=m` driver patches.** `make modules` on single driver dir saves 15-20 min/cycle when applicable.
- **(F) Parallel multi-device testing.** Every owned validation device runs the autonomous cycle concurrently with separate output dirs. Catches per-device defects one cycle earlier.
- **(G) Subagent scope ≤30 min + worktree isolation + single-responsibility.** Empirical pattern: 8-task subagents stall, 2-3-task subagents complete.
- **(H) Lock-file + stale-process hygiene.** Clean `.git/index.lock` + orphan processes on session start. Disable auto git-`gc` in concurrent multi-agent repos.
- **(I) Cycle telemetry per §11.4.24.** Every cycle logs commit, per-phase wall-clock, speedup flag set, outcome. Weekly aggregation surfaces empirical ROI per discipline.

Composes with §11.4.4 / §11.4.6 / §11.4.9 / §11.4.20 / §11.4.24 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.51 / §11.4.52 / §11.4.58 / §11.4.70 / §12.7 / §107. Pre-build gate `CM-ITERATION-SPEEDUP-DISCIPLINE` (when implemented) + paired §1.1 mutation. No escape hatch — no `--skip-phase1`, `--no-pre-flight`, `--rebuild-everything`, `--unlimited-subagent-scope`, `--ignore-locks`, `--no-telemetry` flag exists.

Canonical authority: `Constitution.md` §11.4.82.

Non-compliance is a release blocker.

§11.4.83 — docs/qa/ end-user evidence mandate (User mandate, 2026-05-22)

Direct user mandate (verbatim, 2026-05-22): “every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof.”

Every feature that ships MUST carry a recorded end-to-end communication transcript plus attached materials committed under `docs/qa/<run-id>/`. Transcripts MUST be full bidirectional. Attached materials live in-repo (no external-only links — §11.4.13 sink-side violation). Bot-driven / agent-driven QA MUST preserve

the full conversation thread as the proof artefact. CI release gates MUST refuse to tag a version whose feature-shipping commit lacks its `docs/qa/<run-id>/`.

Composes with §11.4.2, §11.4.5, §11.4.13, §11.4.65, §11.4.69, §107, §1.1.

Canonical authority: `Constitution.md` §11.4.83.

Non-compliance is a release blocker.

§11.4.84 — Working-tree quiescence rule for subagent commits (User mandate, 2026-05-22)

Short tag: `working-tree quiescence`.

Direct user mandate (verbatim, 2026-05-22): “no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add`, the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT.”

No subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Pre-`git add`: grep for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcuts, `// MUTATION` annotations, `_mutated_*` filenames). Cross-check `git status --porcelain` against declared scope → unaccounted entries ABORT.

Lesson (forensic). A consuming project’s logo-fix subagent (Herald commit `72e81ab`, 2026-05-21) swept a `// always pass` JWT-bypass mutation residue into an unrelated commit, pushed to all four mirrors before being caught. Fix `d5bd360` landed within the hour, but the security-defect window is real and the lesson is constitutional.

Operative rule. (1) Pre-`git add` grep + scope-match → unaccounted ABORT. (2) Active mutation gates MUST be serialised before unrelated commits. (3) Concurrent subagents in same checkout MUST use lockfile (`.git/MUTATION_IN_PROGRESS`) or `git worktree add`. (4) Pre-push `mutation-residue-scanner` BLOCKS pushes containing mutation markers.

Composes with §1.1, §11.4.20, §11.4.70, §11.4.27, §11.4.10, §11.4.71, §107.

Canonical authority: `Constitution.md` §11.4.84.

Non-compliance is a release blocker.

§11.4.85 — Stress + Chaos Test Mandate (User mandate, 2026-05-24)

Short tag: `stress-chaos-mandate`.

Direct user mandate (verbatim, 2026-05-24): “Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!”

Every fix MUST ship with full-automation stress + chaos test suites. Happy-path-only coverage is a §11.4 / §107 PASS-bluff at the resilience layer.

Stress = sustained load (N ≥ 100 iters OR ≥ 30 s) + concurrent contention (N ≥ 10 parallel) + boundary conditions (empty/max/off-by-one). **Chaos** = failure-injection per §11.4.69 closed-set (process-kill, network-drop, input-corruption, OOM, disk-full, state-corruption) + verifiable recovery.

Anti-bluff: every stress + chaos PASS cites a captured-evidence artefact (latency.json / categorised_errors.txt / state_delta_snapshot.json) per §11.4.5 + §11.4.69. Helper library `stress_chaos.sh` provides `ab_stress_run` / `ab_stress_concurrent` / `ab_chaos_*` primitives. Chaos cleanup non-negotiable — corrupt-restore / disk-fill-cleanup / process-restart MUST run in trap EXIT.

4-layer per §11.4.4(b): pre-build gate (test files exist + parse) + paired meta-test mutation + on-device test (if LIVE_ADB_TESTABLE) + HelixQA Challenge entry. Composes with §11.4 / §11.4.1 / §11.4.5 / §11.4.6 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.83.

Canonical authority: `Constitution.md` §11.4.85.

Non-compliance is a release blocker. No `--skip-stress`, `--no-chaos`, `--happy-path-suffices` flag exists.

§11.4.86 — Roster/corpus-backed Status-doc auto-sync mandate (User mandate, 2026-05-25)

Forensic anchor (verbatim): “Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!”

Status docs (§11.4.45) backed by a **tracked roster** (installed apps/components) or **asset corpus** (test/media directory) MUST resync out of the box the moment a member changes — Status doc + Status_Summary + HTML + PDF, mechanically. Mechanism (all must hold): (1) drift-proof **fingerprint** = sha256 of sorted member list (NOT mtime), persisted in a sidecar; (2) **sync helper** regenerates fingerprint + re-exports (via §11.4.65); (3) **pre-build gate** FAILs on fingerprint drift (mirrors §11.4.12 + §11.4.45); (4) paired §1.1 mutation. Composes with §11.4.12 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.6. Universal (§11.4.17) — consuming project supplies the specific docs/roster/helper/gate per §11.4.35.

Canonical authority: `Constitution.md` §11.4.86.

Non-compliance is a release blocker. No `--skip-roster-sync`, `--allow-status-drift` flag exists.

§11.4.87 — Endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing mandate (User mandate, 2026-05-26)

When operator instructs an AI agent to “continue in endless loop fully autonomously” (or semantically-equivalent), the agent MUST treat as HARD-CONTRACT covenant: (A) continue until `docs/Issues.md` non-terminal Status entries = 0 AND `docs/CONTINUATION.md` §3 Active work empty AND no subagent in-flight AND no external dep in-flight; (B) dispatch background subagents for parallelisable work — main + subagents concurrent, “waiting for results” is the ONLY idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence “physical proofs” (audio/video/network/UI/sysfs) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS are critical defects; (D) §11.4 anti-bluff covenant family operative end-to-end (tests AND HelixQA Challenges bound equally per forensic anchor “tests pass but features don’t work”); (E) loop terminates ONLY on all-conditions-met, explicit operator STOP, host-safety demand, or scheduled wake on known-future-actionable signal.

Composes with §11.4 / §11.4.1 / §11.4.2 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.7 / §11.4.20 / §11.4.27 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.58 / §11.4.68 / §11.4.69 / §11.4.70 / §11.4.83 / §11.4.85 / §11.4.86 / §12.10. Pre-build gate `CM-COVENANT-114-87-PROPAGATION` + paired §1.1 mutation.

Canonical authority: `Constitution.md` §11.4.87.

Non-compliance is a release blocker. No `--idle-OK`, `--skip-endless-loop`, `--bluff-permitted-for-this-task`, `--metadata-only-test-suffices`, `--no-physical-proof-required` flag exists.

Companion documents

- Constitution.md — authoritative universal Constitution. ALWAYS the tie-breaker.
- CLAUDE.md — Claude Code agent's view of the universal constitution.
- AGENTS.md — OpenCode / Cursor / generic-tooling view.
- README.md — high-level overview + multi-mirror contract.

When operating in a consuming project, ALSO read the project's local QWEN.md / CLAUDE.md / AGENTS.md for project-specific extensions; these MUST NOT weaken any universal rule but MAY add stricter project-specific constraints.

§11.4.88 — Background-push mandate: commit-lock release immediately after commit, push runs detached (User mandate, 2026-05-26)

Forensic anchor: 2026-05-26 commit_all.sh held flock ~5h on synchronous post-commit push. Mandate: (A) flock release IMMEDIATELY after commit; (B) push detached via nohup + disown; (C) push_all.sh per-remote flock — same-remote serializes, different-remote parallel; (D) failures → qa-results/push_failures/, autonomous loop checks per §11.4.87(A); (E) --sync-push escape for §11.4.41 force-push only. Composes §2.1 / §9.2 / §11.4.41 / §11.4.42 / §11.4.71 / §11.4.87. Gates CM-COVENANT-114-88-PROPAGATION + CM-BACKGROUND-PUSH-WIRED + paired §1.1 mutations.

Canonical authority: Constitution.md §11.4.88. Non-compliance is a release blocker.

§11.4.89 — Background test execution mandate (User mandate, 2026-05-27)

Forensic anchor 2026-05-27: conductor invoked long pre_build synchronously, blocking main stream 6-7 min on §JV/§JW/§JX/§JY scaffolding. Symmetric to §11.4.88 at test-execution layer. Mandate: (A) long tests (>30s) run via nohup ... > <log> 2>&1 & + disown; (B) main stream proceeds to §11.4.42 priority queue immediately; (C) hard-dependency gating via poll/pgrep — surface §11.4.66 options if still running; (D) failures land in log files; (E) foreground only <30s OR operator authorisation; (F) per-script flock. Composes §11.4.42 / §11.4.66 / §11.4.82 / §11.4.84 / §11.4.85 / §11.4.87 / §11.4.88. Gates CM-COVENANT-114-89-PROPAGATION + CM-BACKGROUND-TEST-EXECUTION-WIRED + paired §1.1 mutations.

Canonical authority: Constitution.md §11.4.89. Non-compliance is a release blocker.

§11.4.90 — Obsolete status + obsolescence audit (User mandate, 2026-05-27)

§11.4.15 Status closed-set + terminal `Obsolete (→ Fixed.md)`. Reasons: `superseded-by-design-change | superseded-by-later-mandate | feature-removed | duplicate-of | unsupported-topology`. `**Obsolete-Details:**` line mandatory (Since + Reason + Superseding-item + Triple-check evidence). Colorizer `cell-status-obsolete` = light-gray + strikethrough. Audit cadence per §11.4.40 + §11.4.42. Triple-check non-negotiable.

Canonical authority: Constitution.md §11.4.90. Release blocker.

§11.4.91 — Summary-doc clarity (User mandate, 2026-05-27)

Every summary entry's description: self-contained, meaningful, ≥ 6 words / ≥ 40 chars, SUBJECT + PROBLEM/GOAL. Anti-patterns forbidden: section labels, bare metadata, §-letter alone. Generators extract from H1/H2 heading. Pre-build gate `CM-SUMMARY-CLARITY-DESCRIPTIONS`.

Canonical authority: Constitution.md §11.4.91. Release blocker.

§11.4.92 — Multi-pass change-evaluation (User mandate, 2026-05-27)

5-pass evaluation BEFORE commit-ready: (1) Main-task verification; (2) Regression-blast-radius; (3) Cross-feature interaction; (4) Deep-research per §11.4.8; (5) Anti-bluff confirmation. Doc per pass. Trivial exemption: typo/revision/MD-export ONLY if zero source touched.

Canonical authority: Constitution.md §11.4.92. Release blocker.

§11.4.93 — SQLite-SSoT for workable items (User mandate, 2026-05-27)

`docs/.workable_items.db` SQLite — DB authoritative, MD derived. Schema: items + item_history + obsolete_details + operator_block_details + firebase_metadata + meta. Go binary `cmd/workable-items/` sync md↔db + diff + validate + add + close. Bidirectional regen byte-identical. Anti-bluff: unit+integration+stress+chaos per §11.4.85. Go binary in constitution submodule per §11.4.74. 6-phase migration §LA.

Canonical authority: `Constitution.md` §11.4.93. Release blocker.

§11.4.94 — Zero-idle priority-first parallel-by-default (User mandate, 2026-05-27)

Binds §11.4.20/42/58/70/72/82/87/88/89 — always-on. Idle ONLY: externally blocked, operator STOP, §12 host-safety. Before any wake/sleep survey parallel-work queue. Priority MANDATORY (§11.4.72 audio first). Subagent-driven default non-trivial. Background default >30s. Stability per §11.4.92 + §11.4.84 + §12.6-9. Updates at milestones.

Canonical authority: `Constitution.md` §11.4.94. Release blocker.

§11.4.95 — Workable-items DB TRACKED in git (User mandate, 2026-05-27)

§11.4.93 amendment: `docs/workable_items.db` TRACKED, NEVER gitignored. Authoritative source. Every mutation → stage+commit+push DB + MD. WAL-checkpoint before commit. §11.4.77 does NOT apply. Pre-build gates `CM-COVENANT-114-95-PROPAGATION` + `CM-WORKABLE-ITEMS-DB-TRACKED`.

Canonical authority: `Constitution.md` §11.4.95. Release blocker.

§11.4.96 — Safe-parallel-work-with-long-build (User mandate, 2026-05-27)

SAFE during AOSP build: (A) docs/MD; (B) scripts/; (C) pre-build gates; (D) on-device tests; (E) constitution edits+push; (F) submodule push per §11.4.88; (G) read-only ADB probes; (H) subagent dispatch; (I) web research; (J) workable-items DB ops; (K) pre-build + meta-test execution backgrounded.

UNSAFE: (α) git checkout/reset on source; (β) mass deletes/renames under built source; (γ) APK-built submodule pointer; (δ) out/; (ε) make clean; (ζ) container destruction; (η) disk-fill; (θ) §12 host-safety.

Conductor dispatches ≥1 (A)-(K) per pause. “Build running, nothing else to do” NEVER true.

Canonical authority: `Constitution.md` §11.4.96. Release blocker.

§11.4.97 — Max-use-of-idle + progress-update cadence (User mandate, 2026-05-27)

1. Every minute progressable idle = §11.4.97 violation, dispatch CONTINUOUSLY; (B) 1-line operator updates at commit/

subagent/anchor/evidence/migration — no prompt; (C) continuous physical-proof gathering per §11.4.5/6/69; (D) composes with operating-mode anchor family; (E) idle-only-when-blocked closed-set unchanged from §11.4.94(A). Pre-build gates `CM-COVENANT-114-97-PROPAGATION` + `CM-IDLE-TIME-AUDIT`.

Canonical authority: `Constitution.md` §11.4.97. Release blocker.

§11.4.98 — Full-Automation Anti-Bluff (User mandate, 2026-05-28)

Forensic anchor: “Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! … No bluff is allowed!”

Closes the manual-intervention gap §11.4+85+87+89+94 didn’t forbid. (A) every test MUST self-drive end-to-end — PASS/FAIL/SKIP-with-reason without further human action; (B) single exception = one-time credential bootstrap OUTSIDE test execution (.env, ~/.bashrc, OAuth, MTProto-session-activation); (C) concrete live requirements: (1) no “operator MUST type” prompts — drive via MTProto/real-user-API/webhook-fixture/in-process loopback; (2) no UUIDs colliding with active dev session (Herald 2026-05-28: silent exit -1 lesson); (3) no 60s human-response windows (§11.4.50 violation); (4) re-runnability `-count=3` with self-cleaning state; (5) audit COMPLIANT vs NON-COMPLIANT; (6) no false-positive PASS — silent-skip-as-PASS + stale-evidence forbidden, §11.4.3 SKIP-with-reason correct; (D) composes §11.4.85+89+87+94 = continuously-validated fully-automated non-flake anti-bluff regime; (E) inheritance per §11.4.35 — restate literal `11.4.98` in every consumer; pre-build gate `CM-COVENANT-114-98-PROPAGATION`; paired §1.1 mutations strip → FAIL; (F) enforcement: manual-action commit BLOCKED; NON-COMPLIANT after 30 days → §11.4.90 Obsolete citing §11.4.98.

Canonical authority: `Constitution.md` §11.4.98. Release blocker.

§11.4.99 — Latest-Source Documentation Cross-Reference (User mandate, 2026-05-28)

Forensic: “ALWAYS check against latest versions of services we use web/online docs before creating instructions! … mandatory rules / constraints, result is consistency and safety of created instructions, guides and manuals!”

Case study (Herald 2026-05-28): MTProto guide recommended VoIP + omitted `recover@telegram.org` pre-login email; contradicted official Telegram + gotd/td docs; could have caused permanent ban. Misguidance-by-stale-docs = §11.4 PASS-bluff at documentation layer.

1. Pre-commit cross-reference: fetch LATEST official online docs (WebFetch/MCP/direct browsing — NEVER training data) for every operator-facing instruction doc; cite source URL+date in "## Sources verified" footer AND commit-message footer; seek secondary authoritative sources for sparse official docs. (B) Negative findings documented explicitly. (C) STALE after 6 months (90 days for risk-classified services); re-verify at vN.0.0, on breaking-change, on operator-error. (D) Risk-classified: messengers/cloud/payment/AI/code-hosting/package-managers. (E) Composes with §11.4.92 Pass 4 INDEPENDENT — cannot substitute. (F) Inheritance per §11.4.35 — restate literal `11.4.99` in every consumer's CLAUDE/AGENTS/QWEN. (G) Enforcement: missing-footer BLOCKED; stale-beyond-grace → §11.4.90 Obsolete.

Canonical authority: `Constitution.md` §11.4.99. Release blocker.

§11.4.100 — RETIRED. Demoted to consumer project (ATMOSphere video-color/visual-quality fidelity) per §11.4.17/§11.4.35 — project-specific (RK3588/MPV/Arvus), not universal. See the consuming project's Constitution/CLAUDE/AGENTS/QWEN.

§11.4.101 — Autonomous-decision-over-blocking (User mandate, 2026-05-28)

Forensic: "when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven't answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!"

In autonomous / endless-loop mode (per §11.4.87) the agent MUST minimize operator-blocking and make the safe, reliable, reversible decision itself — work NOT stalled overnight waiting for input. §11.4.87 = keep working; §11.4.101 = HOW to clear the decision points.

Decision rule (proceed autonomously when ALL): (a) reversible OR pre-op backup per §9.2; (b) safe choice determinable from captured evidence per §11.4.6 (`LIKELY` ≠ determination); (c) wrong-choice blast radius bounded AND recoverable; (d) composes with §11.4 + §12 + §9. Block-only-when (BLOCK via §11.4.66 ONLY when ALL): irreversible AND high-blast-radius

AND choice undeterminable from evidence — external-account state, inaccessible hardware, destructive-op-without-backup, force-push (§9.2 + §11.4.41), spending / third-party send. `Operator-blocked` per §11.4.21 only after this fires + self-resolution-exhaustion audit. Maximize-progress-while-blocked: a block parks one work unit, not the loop — keep progressing NON-blocked items per §11.4.87 + §11.4.94; pose-and-idle = §11.4.94 + §11.4.97 violation. Composes §11.4.6/21/40/41/66/87/94/§9.2/§12. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-101-PROPAGATION` (literal `11.4.101`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.101. Release blocker. No `--always-block-on-decision` / `--never-decide-autonomously` / `--skip-decision-rule` / `--block-without-self-resolution` escape.

§11.4.102 — Systematic-debugging always-on + always-loaded skill-discovery + plugin availability (User mandate, 2026-05-29)

Forensic: “ALWAYS trigger / start the”/superpowers:systematic-debugging” skills when any issues happen! … we MUST activate the skill(s) and make strongest efforts in full in depth analisys / debugging and determine root causes … “/using-superpowers” skill is ALWAYS loaded, applied and used! All dependencies (plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!”

1. On ANY spotted issue/bug/test-failure/gate-failure/regression/misalignment/inconsistency/unexpected-behaviour the agent MUST activate `superpowers:systematic-debugging` (or platform-equivalent) BEFORE proposing/writing/applying any fix — Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST. Four-phase arc: root-cause → pattern → hypothesis (prove/disprove with captured evidence) → implementation (against proven cause only). Guess-and-retry / symptom-patch / re-run-hoping-it-passes (“probably transient/flaky”) without completed investigation = §11.4.102 violation; calling a failure `transient/flaky/intermittent` without captured forensic evidence = simultaneous §11.4.6 + §11.4.7 violation. (B) `superpowers:using-superpowers` (or platform-equivalent skill-discovery) ALWAYS loaded + applied at session start, consulted before any task — if ANY skill could apply (even 1%) it MUST be invoked, never improvised. (C) Every mandated skill plugin/dependency (Claude Code marketplaces or other runtime ecosystems) MUST be installed + loadable BEFORE dependent work; missing plugin blocking a mandated skill = release-blocker. Install: runtime’s own path — Claude Code `/plugin` flow (add marketplace → install → confirm skill in available-skills list); other runtimes’ documented installer.

Anti-bluff: confirm via live capability list (install exit 0 ≠ skill loadable, §11.4.80 lesson). Composes §11.4.4/6/7/8/43/70/82(A)/92. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-102-PROPAGATION` (literal `11.4.102`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.102. Release blocker. No `--skip-systematic-debugging` / `--guess-and-retry-OK` / `--symptom-patch-permitted` / `--skip-skill-discovery` / `--plugin-optional` / `--missing-plugin-is-warning` escape.